



資料結構

Data Structure

HW 01

姓名/學號: 廖彥瑄 110810030

林彥廷 112370219

HW01-Ex1.

implement a calculator application that evaluates mathematical expressions.

Code

```
#include <iostream>
#include <cctype>
#include <cstring>
#include <typeinfo>
#include <string>
#include <cmath>
#include <iomanip>
using namespace std;

// 將 node 寫成 template → 省略第二個 stack
template <typename T>
struct Node
{
    T data ;
    Node<T>* next;
};

// 將 stack 寫成 template → 省略第二個 stack 。Stack<char> 就是字元 stack ,
Stack<double> 就是數字 stack
template <typename T>
class Stack
{
private:
    Node<T>* top;
public:
    Stack() { top = nullptr; } // 初始化 stack

    // Push 操作: 將元素放入 stack
    void push (T ch)
    {
        Node<T>* newNode = new Node<T>;
        newNode->data = ch;
        newNode->next = top;
        top = newNode;
    }
};
```

```

}

// pop 操作: 移除並回傳頂端元素
T pop()
{
    if (isEmpty()) return '\0';
    T ch = top->data;
    Node<T>* temp = top;
    top = top->next;
    delete temp;
    return ch;
}

// Peek 操作: 取得頂端元素但不移除
T peek()
{
    return (top ? top->data : '\0');
}

// 判斷 stack 是否為空
bool isEmpty()
{
    return top == nullptr;
}
};

// 判斷輸入的字元是不是 operator
bool isOperator(char ch) {
    return ch == '+' or ch == '-' or ch == '*' or ch == '/' or ch == '^' or ch ==
'%';
}

// 錯誤處理: 判斷輸入的是不是數字、括號或 operator
bool isValidChar(char ch) {
    return isdigit(ch) or ch == '.' or ch == '+' or ch == '-' or ch == '*' or ch
== '/' or ch == '^' or ch == '%' or ch == '(' or ch == ')';
}

```

// 賦予 operator 不同的優先順序: 次方 > 乘、除與求餘 > 加、減

```
int precedence(char op)
{
    if (op == '^')
    {
        return 3;
    }
    else if (op == '*' or op == '/' or op == '%')
    {
        return 2;
    }
    else if (op == '+' or op == '-')
    {
        return 1;
    }
    else
    {
        return 0;
    }
}
```

// infix 轉 postfix

```
bool infixToPostfix(string infix, string postfix[], int& postfixcount)
{
    Stack<char> s; // 字元 stack
    postfixcount = 0;

    // 遍歷 infix
    for (int i = 0; i < infix.length(); i++) {

        // 錯誤處理: 先判斷 infix[i] 有沒有輸入錯誤
        if (not isValidChar(infix[i])) {
            cout << "Error: Invalid character '" << infix[i] << "'" << endl;
            return false;
        }

        // 如果不是 operator 就加入到 postfix
        if (isdigit(infix[i]) or infix[i] == '.') {
```

```

        string number = "";

        // 繼續遍歷直到非數字或小數點
        while (i < infix.length() and (isdigit(infix[i]) or infix[i] == '.'))
        {
            number += infix[i]; // 打包成 number
            i++;
        }
        i--;
        postfix[postfixcount++] = number; // 把 number 加入到 postfix
    }

    // 如果是左括號
    else if (infix[i] == '(') {

        // 錯誤處理: 判斷是不是空括號
        if (i + 1 < infix.length() and infix[i+1] == ')') {
            cout << "Error: Empty parentheses" << endl;
            return false;
        }
        s.push(infix[i]); // 左括號加到 stack
    }

    // 如果是右括號
    else if (infix[i] == ')') {

        // pop operators 直到左括號
        while (not s.isEmpty() and s.peek() != '(') {
            postfix[postfixcount++] = s.pop();
        }

        // 錯誤處理: 判斷 Stack 裡面是不是空的, 避免只有右括號
        if (s.isEmpty()) {
            cout << "Error: Unmatched closing parenthesis" << endl;
            return false;
        }

        s.pop(); // pop 左括號
    }
}

```

```

    }

    // 負數處理：讀到 '-' 且位置在一開始或前一個字元是左括號或 operator
    else if (infix[i] == '-' and (i == 0 or infix[i-1] == '(' or infix[i-1]
    == '+' or infix[i-1] == '-' or infix[i-1] == '*' or infix[i-1] == '/' or infix[i-
    1] == '^' or infix[i-1] == '%')) {
        string number = "-";
        i++; // 從負數下一個字元開始處理

        // 繼續遍歷直到非數字或小數點
        while (i < infix.length() and (isdigit(infix[i]) or infix[i] == '.'))
    {
        number += infix[i]; // 打包成 number
        i++;
    }
    i--;
    postfix[postfixcount++] = number; // 把 number 加入到 postfix
}

// 如果是 operator
else if (isOperator(infix[i])) {

    // 錯誤處理：結尾是 operator
    if (i == infix.length() - 1) {
        cout << "Error: Expression ends with operator" << endl;
        return false;
    }

    // 錯誤處理：如果連續兩個 operators
    if (i > 0 and isOperator(infix[i-1])) {
        cout << "Error: Consecutive operators" << endl;
        return false;
    }

    // 如果 stack 裡 operator 的 precedence 比 infix 的 operator 還要大，就放到
    postfix 裡
    while (not s.isEmpty() and s.peek() != '(' and precedence(s.peek())
    >= precedence(infix[i])) {

```

```

        postfix[postfixcount++] = s.pop();
    }
    s.push(infix[i]); // 把目前的 operator 放入 stack
}
}

// 錯誤處理: 如果只有輸入左括號卻沒輸入右括號
while (not s.isEmpty()) {
    if (s.peek() == '(') {
        cout << "Error: Unmatched opening parenthesis" << endl;
        return false;
    }
    postfix[postfixcount++] = s.pop();
}

// 錯誤處理: 如果 postfix 是空的
if (postfixcount == 0) {
    cout << "Error: Empty expression" << endl;
    return false;
}

return true; // 都沒錯→回報 true 繼續
}

// 輸出 postfix
void printPostfix (string postfix[], int postfixcount)
{
    cout << "Postfix expression: ";
    // 遍歷 postfix
    for (int i = 0; i < postfixcount; i++)
    {
        cout << postfix[i];
    }
    cout << endl;
}

bool evaluatePostfix(string postfix[], int postfixcount)
{

```

```

Stack<double> evaluation; // 浮點數 stack

// 逐步顯示計算過程
cout << "\n--- Step-by-step Evaluation ---" << endl;

// 遍歷 postfix
for (int i = 0; i < postfixcount; i++)
{
    string token = postfix[i]; // 把目前的 element 作為 token

    // 如果 token 長度為 1 並且是 operator 則開始處理計算
    if (token.length() == 1 and isOperator(token[0]))
    {
        // 錯誤處理: 避免只有 operator 沒有 operand
        if (evaluation.isEmpty()) {
            cout << "Error: Missing operands" << endl;
            return false;
        }

        double a = evaluation.pop(); // pop 頂層 operand

        // 錯誤處理: 避免 pop a 後只有一個 operand
        if (evaluation.isEmpty()) {
            cout << "Error: Missing operands" << endl;
            return false;
        }

        double b = evaluation.pop(); // pop 第二層 operand

        cout << "Pop: " << b << ", Pop: " << a << " -> " << b << " " <<
token[0] << " " << a; // 印出計算過程

        // 判斷 token[0] 是哪種 operator
        if (token[0] == '+')
        {
            evaluation.push(b + a); // b+a
        }
        else if (token[0] == '-')

```



```

    {
        evaluation.push(b - a); // b-a
    }
    else if (token[0] == '*')
    {
        evaluation.push(b * a); // b*a
    }
    else if (token[0] == '/')
    {
        // 錯誤處理: 避免分母為0
        if (a == 0) {
            cout << "Error: Division by zero" << endl;
            return false;
        }
        evaluation.push(b / a); // b/a
    }
    else if (token[0] == '^')
    {
        evaluation.push(pow(b, a)); // b^a
    }
    else if (token[0] == '%')
    {
        // 錯誤處理: 避免除數為0
        if (a == 0) {
            cout << "Error: Modulo by zero" << endl;
            return false;
        }
        evaluation.push(fmod(b, a)); // b%a
    }

    cout << " = " << evaluation.peek() << " -> Push: " <<
evaluation.peek() << endl; // 印出計算結果
}
// 如果 token 不是 operator
else {
    evaluation.push(stod(token)); // 把 token 轉為 double 存入 stack
    cout << "Push: " << token << endl;
}

```

```

}

// 把 evaluation 存入 result
double result = evaluation.pop();

// 錯誤處理: 避免 evaluation 裡還有東西
if (not evaluation.isEmpty()) {
    cout << "Error: Expression incomplete" << endl;
    return false;
}

// 印出計算結果
cout << fixed << setprecision(1); // 控制輸出格式: 5 → 5.0
cout << "-----" << endl;
cout << "Result: ";
cout << result << endl;
return true;
}

int main()
{
    string infix, postfix[100];
    int postfixcount = 0;

    cout << "Enter an Infix expression: ";
    getline(cin, infix); // 輸入中序表達式。getline: 避免輸入空白導致錯誤

    // 假設 infixToPostfix 沒有問題
    if (infixToPostfix(infix, postfix, postfixcount)) {
        printPostfix(postfix, postfixcount); // 輸出後序表達式
        evaluatePostfix(postfix, postfixcount); // 計算 postfix 結果
    }

    return 0;
}

```

Result

```
C:\Users\User\Desktop\資料結構\HW01\Untitled1.exe
Enter an Infix expression: 3+4-2
Postfix expression: 34+2-

--- Step-by-step Evaluation ---
Push: 3
Push: 4
Pop: 3, Pop: 4 -> 3 + 4 = 7 -> Push: 7
Push: 2
Pop: 7, Pop: 2 -> 7 - 2 = 5 -> Push: 5
-----
Result: 5.0

-----
Process exited after 76.55 seconds with return value 0
請按任意鍵繼續 . . .
```

```
C:\Users\User\Desktop\資料結構\HW01\Untitled1.exe
Enter an Infix expression: 2*3+4
Postfix expression: 23*4+

--- Step-by-step Evaluation ---
Push: 2
Push: 3
Pop: 2, Pop: 3 -> 2 * 3 = 6 -> Push: 6
Push: 4
Pop: 6, Pop: 4 -> 6 + 4 = 10 -> Push: 10
-----
Result: 10.0

-----
Process exited after 2.432 seconds with return value 0
請按任意鍵繼續 . . . ■
```

```
C:\Users\User\Desktop\資料結構\HW01\Untitled1.exe
Enter an Infix expression: 2*(3+4)
Postfix expression: 234+*

--- Step-by-step Evaluation ---
Push: 2
Push: 3
Push: 4
Pop: 3, Pop: 4 -> 3 + 4 = 7 -> Push: 7
Pop: 2, Pop: 7 -> 2 * 7 = 14 -> Push: 14
-----
Result: 14.0

-----
Process exited after 2.614 seconds with return value 0
請按任意鍵繼續 . . . ■
```

C:\Users\User\Desktop\資料結構\HW01\Untitled1.exe

```
Enter an Infix expression: 10/2+3
Postfix expression: 102/3+

--- Step-by-step Evaluation ---
Push: 10
Push: 2
Pop: 10, Pop: 2 -> 10 / 2 = 5 -> Push: 5
Push: 3
Pop: 5, Pop: 3 -> 5 + 3 = 8 -> Push: 8
-----
Result: 8.0

-----
Process exited after 1.828 seconds with return value 0
請按任意鍵繼續 . . . █
```

C:\Users\User\Desktop\資料結構\HW01\Untitled1.exe

```
Enter an Infix expression: 2^3+1
Postfix expression: 23^1+

--- Step-by-step Evaluation ---
Push: 2
Push: 3
Pop: 2, Pop: 3 -> 2 ^ 3 = 8 -> Push: 8
Push: 1
Pop: 8, Pop: 1 -> 8 + 1 = 9 -> Push: 9
-----
Result: 9.0

-----
Process exited after 1.435 seconds with return value 0
請按任意鍵繼續 . . . █
```

C:\Users\User\Desktop\資料結構\HW01\Untitled1.exe

```
Enter an Infix expression: 25%7+10
Postfix expression: 257%10+

--- Step-by-step Evaluation ---
Push: 25
Push: 7
Pop: 25, Pop: 7 -> 25 % 7 = 4 -> Push: 4
Push: 10
Pop: 4, Pop: 10 -> 4 + 10 = 14 -> Push: 14
-----
Result: 14.0

-----
Process exited after 1.342 seconds with return value 0
請按任意鍵繼續 . . . █
```

C:\Users\User\Desktop\資料結構\HW01\Untitled1.exe

Enter an Infix expression: (2+3)*(4-1)

Postfix expression: 23+41-*

--- Step-by-step Evaluation ---

Push: 2

Push: 3

Pop: 2, Pop: 3 -> $2 + 3 = 5$ -> Push: 5

Push: 4

Push: 1

Pop: 4, Pop: 1 -> $4 - 1 = 3$ -> Push: 3

Pop: 5, Pop: 3 -> $5 * 3 = 15$ -> Push: 15

Result: 15.0

Process exited after 1.262 seconds with return value 0

請按任意鍵繼續 . . .

C:\Users\User\Desktop\資料結構\HW01\Untitled1.exe

Enter an Infix expression: 2.5+3.5*2

Postfix expression: 2.53.52*+

--- Step-by-step Evaluation ---

Push: 2.5

Push: 3.5

Push: 2

Pop: 3.5, Pop: 2 -> $3.5 * 2 = 7$ -> Push: 7

Pop: 2.5, Pop: 7 -> $2.5 + 7 = 9.5$ -> Push: 9.5

Result: 9.5

Process exited after 1.165 seconds with return value 0

請按任意鍵繼續 . . .

一、編譯與執行方法

使用 Dev-C++ 編譯與執行

二、使用的資料結構

我們用 linked list 實作 stack，並透過 template 讓同一套程式碼能產生兩種型別的 stack。第一種是 `Stack<char>`，用來存放運算子（`+`、`-`、`*`、`/`、`^`、`%`）和左括號（`（`）。在 infix 轉 postfix 的過程中，根據運算子優先順序決定要 push 進 stack 還是從 stack 中 pop 出來放到 postfix。

第二種是 `Stack<double>`，用於計算 postfix expression。遇到數字時轉成 double 後 push 進 stack，遇到運算子時 pop 出兩個數字做運算，再把結果 push 回去。走訪完後 stack 頂端即為答案。

Postfix expression 本身用 string 陣列儲存，因為運算中可能出現多位數、負數和小數，每個 token 以一個完整的 string 表示，避免多位數被拆成單一字元的問題。

三、infix to postfix 轉換演算法

我們用 stack 做轉換，程式依序掃描 infix expression 的每個字元，根據字元類型做不同處理。

- ✧ 數字：先判斷它是不是多位數或小數，如果是的話用 while 迴圈往後讀取連續的數字或小數點，串成一個完整的數字字串後，放入 postfix。
 - ✧ 左括號：直接存進 stack
 - ✧ 右括號：程式會一直從 stack 中拿出運算子，放進 postfix expression 直到遇到左括號，然後再把左括號拿掉。
 - ✧ 運算子：程式會比較這個運算子和 stack 裡最後進去的運算子的優先順序，如果最後進去的運算子優先順序較高或是相同，就會把最後進去的運算子拿到 postfix expression，然後再把目前讀到的運算子放進 stack。
 - ✧ 結束收尾：infix 掃描完畢後，將 stack 中剩餘的運算子全部 pop 到 postfix。
- 此外，程式也處理負號：當 `-` 出現在表達式開頭、左括號後方或另一個運算子後方時，視為負號而非減法，將 `-` 與後續數字合併成一個 token（如 `"-3"`）。

運算子	優先順序
<code>^</code>	高
<code>*</code> <code>/</code> <code>%</code>	中
<code>+</code> <code>-</code>	低

圖一 運算子優先順序

四、時間複雜度分析

在 infix-to-postfix 轉換中，程式會從左到右看整個運算式。每個數字、括號或運算子最多被讀一次，運算子會被放進 stack 一次還有被拿出來一次。所以整個轉換過程的時間複雜度是 $O(n)$

在 postfix evaluation 中，程式會看整個 postfix expression。遇到數字時會放進 stack，遇到運算子時會把兩個數字做計算，再把結果放回去。所以計算 postfix expression 的時間複雜度是 $O(n)$ ，從以上兩個分析可以知道整個程式的總時間複雜度是 $O(n)$

五、遇到的挑戰

挑戰 1：Stack 型別設計

- 問題：一開始 Stack 只存 char，求值時結果被迫轉回 ASCII，超過 9 就算不出來
- ✓ 解法：改成 `template <typename T>`，一個 class 同時給 `Stack<char>`（存運算子）和 `Stack<double>`（存數字）用

挑戰 2：多位數與浮點數解析

- 問題：原本一個字元一個字元讀，"12" 被拆成 '1' '2'，無法辨識完整數字
- ✓ 解法：改用 string 陣列存 postfix，掃描時用 while 連續讀取，直到非數字非小數點為止，整包當一個 token

挑戰 3：負號與減法混淆

- 問題：- 在表達式開頭、(後面、運算子後面是負號，其他位置是減法
- ✓ 解法：判斷 `i == 0` 或前一字元是 (或運算子時，將 - 和後續數字打包成一個 token（如 "-3"）

挑戰 4：後序求值的運算元順序

- 問題：減法和除法的 pop 順序會影響結果，先 pop 的是右運算元，後 pop 的是左運算元
- ✓ 解法：明確區分 `a = pop()`（右）、`b = pop()`（左），運算時寫成 `b - a`、`b / a`

六、可以改進的地方

這次因為時間的關係我們只有些完成一些基本功能，但沒有加入 logical operators、comparison operators、bitwise operations 這些功能，讓程式可以做更複雜的運算。這次作業讓我們發現時間管理的重要性，下次希望可以更早開始做作業，才有充裕的時間做出更多進階功能，讓程式更完整。

附錄：AI 工具使用紀錄

AI 工具：Claude Code CLI

摘要：

對話範圍	AI 輔助內容	我的修改
evaluatePostfix 錯誤	指出 char 存整數的 ASCII 問題	自己改成 Stack<int> 再改成 Stack<double>
多位數/浮點數	提出用 string[] 解決 token 問題	參考概念後自己重寫
Template	說明 template 寫法	自己改成 template
次方、求餘 operator	提示優先權與實作位置	自己加進 precedence 和 evaluatePostfix
負數處理	寫了負號判斷邏輯	理解後自己微調
錯誤處理	寫了所有錯誤檢查	發現並簡化多餘的) 檢查
逐步顯示	加了 step-by-step 輸出	直接採用，調整格式
註解	校對	自己寫全部註解

Stack 與 linked list 結構：完全自己實作

中序轉後序演算法：自己寫，AI 協助 debug

後序求值演算法：自己寫，AI 協助 debug

負數處理、錯誤處理、顯示計算過程：AI 寫，自己修正

部分對話：

我：我現在想測試完運算->數字 stack->雙位數計算...，如何？

LLM：

你的順序方向沒錯，但有個相依性的問題——測試運算之前，你得先有數字 stack，否則減法和除法會因為 char 型別的限制測不出正確結果。

我先問一個問題：關於數字 stack，你偏好哪種做法？

選項 1：Copy-Paste 改型別（最簡單）

選項 2：Template（稍微進階）

選項 3：先用 vector<double> 頂（偷懶）

我：我改了一下成功了，但目前的寫法是不是還得擴展到小數，雙位數

LLM：

順序建議

1. 先把 `Stack<int>` 改成 `Stack<double>`，確認個位數 `.5*2` 這種能跑
 2. 再改 `infixToPostfix` 的數字掃描，讓它產出帶分隔符的 postfix（例如 `12 3+`）
 3. 最後改 `evaluatePostfix` 的數字讀取，讓它能從分隔符之間解析出完整數字
-

我：string 陣列如何解決 char 陣列問題

LLM：

postfix 是 `char[100]`，一個字元一個字元塞

`"12+3"` → postfix = `['1', '2', '+', '3', '\0']`

↑ 1 和 2 分家，讀的時候不知道是 `"12"` 還是 `"1" "2"`

postfix 是 `string[100]`，每個 token 是一個完整的字串

`"12+3"` → postfix[0] = `"12"`

postfix[1] = `"+"`

postfix[2] = `"3"`

↑ `"12"` 被當成一個單位存

做法拆成三步：

1. 掃描時（line 91-95）：看到數字就 while 一路往後讀，`number += infix[i]` 把字元串起來，直到碰到非數字非小數點為止。整串 `"12"` 當一個 string 存進 `postfix[]`。
2. token 本身就是完整單位（line 132）：`evaluatePostfix` 用 `postfix[i]` 拿到的是整個 `"12"`，不是 `'1'` 和 `'2'` 分兩次。
3. 轉數字（line 142）：直接用 `stod(token)` 把 `"12"` 轉成 `double`，一行搞定，不需要 `-'0'` 也不怕多位數。